

ESPECIFICACIÓN DE INTEGRACIÓN

swagger_consulta_lineas.yaml

A23

Propósito: responder, en el momento de evaluar una oferta, **cuánto cupo hay disponible** en los tres niveles que la regla de validación compara. Es la API que alimenta el motor de asignación de líneas ([Specs_Procesos/Lineas/spec-asignacion-lineas.md](#)).

ENDPOINT	USO
POST /consulta	Cupo de un cliente y de todos los deudores de la operación, en los tres niveles

Frecuencia: bajo demanda, **una sola llamada por evaluación** con todos los RUT deudores. No una llamada por deudor: el motor reevalúa la operación completa (§4.2 del spec de líneas) y N llamadas devuelven N snapshots distintos.

Los tres niveles

La regla de validación es $\text{monto_factura} \leq \min(\text{disponible_cliente}, \text{disponible_cliente_deudor}, \text{disponible_deudor})$. La API devuelve exactamente esos tres, y no obliga al consumidor a derivarlos:

NIVEL	QUÉ ES	LÍNEAS QUE AGRUPA	ALCANCE
Cliente	Cupo del cliente	LF1 (inicial) y LF4 (otros deudores) — comodines, sin RUT deudor	Compartido entre todos los deudores de la operación
Cliente-deudor	Cupo del par	LF2 (normal) y LF3 (puntual)	Exclusivo de ese par
Deudor	Exposición máxima del factoring a ese deudor	—	Compartido entre todos los clientes que le ceden, incluidas carteras de otros ejecutivos

El nivel cliente viene UNA vez, no por deudor. LF1 y LF4 son pozos comodín: repetirlos por deudor y sumarlos duplica cupo que no existe.

Los cuatro montos

En los tres niveles y en cada línea individual:

`disponible = aprobada - utilizada - reservada`

CAMPO	SIGNIFICADO
<code>aprobada</code>	Lo que el comité aprobó
<code>utilizada</code>	Exposición viva: cedido y no pagado. Una factura vencida e impaga sigue consumiendo ; el cupo se libera sólo cuando el deudor paga
<code>reservada</code>	Cupo comprometido por operaciones aceptadas y aún no aprobadas por Operaciones
<code>disponible</code>	Lo que queda para operaciones nuevas

Quién es dueño de la reserva

NEX sólo lee. El ciclo de vida es del sistema de gestión de líneas y lo cierra el core:

1. Mientras el cliente no acepta, lo que existe es una **evaluación**, no una reserva. Este endpoint es consulta y no persiste nada.
2. **El cliente acepta** → el sistema de gestión de líneas **crea la reserva** (`reservada` sube).
3. **Operaciones aprueba en el core** → el core **commitea la reserva**: la elimina y la convierte en línea utilizada (`reservada` baja, `utilizada` sube por el mismo monto).

Consecuencia para el consumidor: una operación ya aceptada no depende de que NEX vuelva a evaluarla para conservar su cupo.

La asignación se calcula sobre el disponible que devuelve esta API

No hay corrección del lado del consumidor. `disponible` es el número con el que se asigna, tal como llega: la evaluación anterior de una operación no reserva cupo, no protege facturas y no entra al cálculo. Lo que el consumidor sí guarda es la **versión de la simulación**, que congela el resultado de cada evaluación como evidencia y permite mostrar qué se movió entre una y otra —una línea ampliada, o cupo consumido por otro negocio cursado por otro canal—. Ver §4.3 del spec de asignación de líneas.

Reglas de la respuesta

- **Snapshot único.** `consultadoEn` vale para los tres niveles. Si se arman desde lecturas de instantes distintos, la comparación `min(...)` cruza estados que nunca coexistieron y deja pasar operaciones sobre cupo inexistente.
- **Línea suspendida.** Conserva su `utilizada` —la suspensión no libera lo ya cedido— pero no admite operaciones nuevas: devuelve `disponible = 0`. No omitir la línea: el consumidor necesita distinguir «suspendida» de «inexistente» para explicar el motivo del rechazo.
- **Par sin línea propia.** Se devuelve igual, con montos en cero y `sinLineaPropia = true`. El motor lo necesita para saber que el único camino de ese deudor es la LF4 del nivel cliente, y que el motivo de un eventual rechazo es `lf4` y no `par`.
- **Elegibilidad expuesta, no en duro.** `soloPrime` (LF1) y `unSoloUso` (LF1, LF3) viajan en la respuesta para que el motor no lleve esas reglas escritas en su código.

- **Tope propio del cliente.** `topePropio = false` significa que el nivel cliente es un consolidado de reporte y **no puede bloquear por sí solo** (si cada par está dentro de su línea, la suma también). Sólo bloquea si el comité asignó un tope inferior a la suma de los cupos.
- `version` por línea, para el control de concurrencia optimista del curse.

Relación con los otros activos de línea

ACTIVO	QUÉ APORTA	EN QUÉ SE DIFERENCIA DE A23
A7 — CSV diario de líneas vigentes	La estructura de las líneas del cliente	Batch, una vez al día; sin nivel deudor y sin reservado
A8 — Montos de líneas	Refresco horario de uso/disponible de las líneas de A7	Por <code>idLinea</code> y por cliente; no trae reservado ni la exposición global del deudor
A23 — Esta API	Cupo al momento de evaluar , en los tres niveles, con reservado	Bajo demanda, una llamada por evaluación

A7/A8 alimentan la vista «Líneas» del menú, que es fotografía de cartera. A23 alimenta la decisión de cursar, que exige el dato fresco y el nivel deudor.

Resiliencia: ante error o timeout, la evaluación **no se completa** y el resultado queda en «Por evaluar». Es preferible a mostrar un cursable calculado con cupos viejos: el ejecutivo compromete plazos de giro sobre esa cifra.

Unidad de los montos

`aprobada`, `utilizada`, `reservada` y `disponible` van en **pesos**, enteros, en los tres niveles.

Se llamaban `aprobadaMM` y compañía y viajaban en millones. Eso obligaba al consumidor a reinflar a pesos para comparar contra el monto de una factura —que sí es exacto— y cada ida y vuelta perdía hasta \$100.000 por línea. El millón es una abreviatura de pantalla; el contrato no la usa.

El cupo que aprueba el comité **puede ser cualquier monto**: típicamente es una cifra redonda, pero el consumidor no debe suponerlo. `disponible = aprobada - utilizada - reservada` se cumple de forma exacta, sin tolerancia.